

Database Guide — Vermont Health Platform (HTR)

Audience: Backend developers, database administrators. **Version:** 4.2.0 **Database:** Supabase PostgreSQL 15 + pgvector + Row Level Security

Table of Contents

- 1. Overview
- 2. Schema — Core Tables
- 3. Schema — RAG Vector Store
- 4. Schema — Content & Academy
- 5. Schema — Community & Social
- 6. Row Level Security Policies
- 7. Database Functions & Triggers
- 8. Hybrid Search Implementation
- 9. Migrations
- 10. Supabase Client Patterns
- 11. Performance & Indexing
- 12. Backup & Recovery

1. Overview

The platform uses **Supabase** — a hosted PostgreSQL service that provides:

- Managed PostgreSQL 15 with connection pooling (PgBouncer)
- pgvector extension for embedding storage and similarity search
- Row Level Security (RLS) for per-row authorization
- Auto-generated REST and GraphQL APIs
- Realtime subscriptions via WebSockets
- Supabase Auth (JWT-based) integrated with all tables

Key Design Principles

- Every protected table has RLS enabled — no table is readable without a valid policy
- Users can only access their own data; admins bypass RLS via service role key
- Vector embeddings are stored alongside their source text in `rag_documents`
- The `user_roles` table supports multi-role assignments (a user can hold `subscriber` and `student` simultaneously)
- All webhook payloads are logged in `webhook_inbox` for idempotent processing

2. Schema — Core Tables

profiles

Extends `auth.users` with display and account metadata.

```
CREATE TABLE public.profiles (  
  id          UUID PRIMARY KEY REFERENCES auth.users(id) ON DELETE CASCADE,  
  email       TEXT,  
  full_name   TEXT,  
  avatar_url  TEXT,  
  org_name    TEXT,  
  bio         TEXT,  
  created_at  TIMESTAMPTZ DEFAULT NOW() NOT NULL,  
  updated_at  TIMESTAMPTZ DEFAULT NOW() NOT NULL  
);
```

Auto-populated: A trigger on `auth.users` inserts a profile row on every new signup.

user_roles

Stores role assignments. A user may hold multiple roles simultaneously.

```
CREATE TABLE public.user_roles (  
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  user_id     UUID NOT NULL REFERENCES auth.users(id) ON DELETE CASCADE,  
  role        public.user_role NOT NULL DEFAULT 'free',  
  granted_at  TIMESTAMPTZ DEFAULT NOW() NOT NULL,  
  expires_at  TIMESTAMPTZ,          -- NULL = never expires  
  granted_by  UUID REFERENCES auth.users(id),  
  UNIQUE(user_id, role)  
);
```

Role enum (`public.user_role`):

Value	Tier	Notes
free	Free	Default on signup
subscriber	Subscriber	Grants full article + AI access
student	Student	Same as subscriber, .edu verified
professional	Professional	Adds Advisory Hub + API access
advisory	Advisory	Custom engagement tier
admin	Admin	Full platform access, bypasses RLS

subscriptions

Tracks Stripe subscription state per user.

```
CREATE TABLE public.subscriptions (  
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  user_id     UUID NOT NULL UNIQUE REFERENCES auth.users(id) ON  
DELETE CASCADE,  
  stripe_customer_id TEXT,  
  stripe_subscription_id TEXT UNIQUE,  
  plan        TEXT NOT NULL DEFAULT 'free',  
  status       public.subscription_status NOT NULL DEFAULT 'active',  
  current_period_start TIMESTAMPTZ,  
  current_period_end   TIMESTAMPTZ,  
  cancel_at_period_end BOOLEAN DEFAULT FALSE,  
  created_at  TIMESTAMPTZ DEFAULT NOW() NOT NULL,  
  updated_at  TIMESTAMPTZ DEFAULT NOW() NOT NULL  
);
```

Status enum (`public.subscription_status`): `active`, `trialing`, `past_due`, `canceled`, `incomplete`, `paused`.

stripe_customers

Maps Supabase user IDs to Stripe customer IDs for billing portal access.

```
CREATE TABLE public.stripe_customers (  
  id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  user_id           UUID NOT NULL UNIQUE REFERENCES auth.users(id) ON DELETE  
  CASCADE,  
  stripe_customer_id TEXT NOT NULL UNIQUE,  
  created_at        TIMESTAMPTZ DEFAULT NOW() NOT NULL  
);
```

stripe_events

Idempotency log for Stripe webhook events. Prevents double-processing on retries.

```
CREATE TABLE public.stripe_events (  
  id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  event_id          TEXT NOT NULL UNIQUE,    -- Stripe event ID (evt_...)  
  event_type        TEXT NOT NULL,  
  payload           JSONB NOT NULL,  
  processed_at      TIMESTAMPTZ DEFAULT NOW() NOT NULL  
);
```

api_keys

API keys for Professional/Advisory subscribers accessing the `/api/v1/*` developer API.

```
CREATE TABLE public.api_keys (  
  id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  user_id           UUID NOT NULL REFERENCES auth.users(id) ON DELETE CASCADE,  
  key_hash          TEXT UNIQUE NOT NULL,    -- SHA-256 hash of the key  
  tier              TEXT DEFAULT 'researcher',  
  requests_today    INTEGER DEFAULT 0,  
  last_used_at      TIMESTAMPTZ,  
  revoked_at        TIMESTAMPTZ,  
  created_at        TIMESTAMPTZ DEFAULT NOW() NOT NULL  
);
```

Keys are stored hashed — the plaintext key is shown to the user once at creation and never stored.

ingest_jobs

Tracks asynchronous RAG index rebuild jobs.

```
CREATE TABLE public.ingest_jobs (  
  id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  status            TEXT CHECK (status IN  
  ('queued','running','completed','failed')),  
  started_at        TIMESTAMPTZ,  
  completed_at      TIMESTAMPTZ,  
  error_message      TEXT  
);
```

webhook_inbox

Receives and deduplicates incoming webhooks (Sanity, Stripe) before processing.

```
CREATE TABLE public.webhook_inbox (  
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  source      TEXT NOT NULL,      -- 'sanity' | 'stripe'  
  event_type  TEXT,  
  payload     JSONB NOT NULL,  
  processed   BOOLEAN DEFAULT FALSE,  
  processed_at TIMESTAMPTZ,  
  created_at  TIMESTAMPTZ DEFAULT NOW() NOT NULL  
);
```

3. Schema — RAG Vector Store

rag_documents

Stores chunked text + embeddings for the AI Analyst's knowledge base.

```
CREATE TABLE public.rag_documents (  
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  source_type TEXT NOT NULL,      -- 'pdf' | 'sanity_post' |  
  'sanity_policy' | etc.  
  source_id   TEXT NOT NULL,      -- Sanity _id or PDF filename  
  title       TEXT,  
  chunk_text  TEXT NOT NULL,  
  chunk_index INTEGER DEFAULT 0,  
  embedding   VECTOR(768),        -- text-embedding-3-small, 768 dims  
  pillar      TEXT,              --  
  'Policy' | 'Economics' | 'Technology' | 'Clinical' | 'Equity'  
  url         TEXT,              -- canonical URL for citations  
  last_indexed_at TIMESTAMPTZ DEFAULT NOW() NOT NULL,  
  metadata    JSONB DEFAULT '{}'::jsonb,  
  UNIQUE(source_id, chunk_index)  
);
```

Indexes:

```
-- HNSW index for approximate nearest-neighbor (ANN) vector search  
CREATE INDEX idx_rag_embedding  
ON public.rag_documents  
USING hnsw (embedding vector_cosine_ops)  
WITH (m = 16, ef_construction = 64);  
  
-- Source lookups (for incremental re-indexing)  
CREATE INDEX idx_rag_source ON public.rag_documents(source_type, source_id);  
  
-- Pillar filtering  
CREATE INDEX idx_rag_pillar ON public.rag_documents(pillar);  
  
-- Full-text search index (BM25 via tsvector)  
CREATE INDEX idx_rag_content_fts  
ON public.rag_documents  
USING gin (to_tsvector('english', chunk_text));
```

rag_query_log

Logs every AI Analyst query for quality monitoring and debugging.

```
CREATE TABLE public.rag_query_log (  
  id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  user_id           TEXT,  
  query             TEXT,  
  role              TEXT,  
  model_used        TEXT,  
  retrieved_doc_ids TEXT[],  
  retrieved_scores   FLOAT[],  
  response_preview   TEXT,  
  latency_ms         INTEGER,  
  was_zero_result    BOOLEAN DEFAULT FALSE,  
  created_at         TIMESTAMPTZ DEFAULT NOW()  
);
```

This table has no RLS — it is only accessible via the service role key (admin dashboard only).

4. Schema — Content & Academy

state_performance_index

Performance Index scores for all 50 U.S. states, displayed in the State Dashboard.

```
CREATE TABLE public.state_performance_index (  
  id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  state_name         TEXT,  
  state_id           TEXT UNIQUE,    -- e.g. 'vermont', 'california'  
  performance_score   FLOAT,  
  cost_index          FLOAT,  
  quality_score       FLOAT,  
  access_score        FLOAT,  
  equity_score        FLOAT,  
  innovation_score    FLOAT,  
  preventive_care_rate FLOAT,  
  uninsured_rate      FLOAT,  
  data_year           INTEGER,  
  updated_at         TIMESTAMPTZ DEFAULT NOW()  
);
```

rht_state_profiles

Rural Health Transformation program data per state, displayed on the State Dashboard RHT tab.

```
CREATE TABLE public.rht_state_profiles (  
  id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  state_id           TEXT UNIQUE,  
  state_name         TEXT,  
  award_amount       TEXT,  
  status             TEXT,  
  strategic_focus     TEXT,  
  description         TEXT,  
  initiatives         JSONB DEFAULT '[]',  -- Array of {title, description,  
status}  
  metrics             JSONB DEFAULT '[]',  -- Array of {label, value, target,  
unit}  
  simulation          JSONB                -- Optional simulation parameters  
);
```

bookmarks

User-saved article bookmarks.

```
CREATE TABLE public.bookmarks (  
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  user_id     UUID NOT NULL REFERENCES auth.users(id) ON DELETE CASCADE,  
  sanity_id   TEXT NOT NULL,  
  slug        TEXT NOT NULL,  
  title       TEXT,  
  pillar      TEXT,  
  content_type TEXT,  
  created_at  TIMESTAMPTZ DEFAULT NOW() NOT NULL,  
  UNIQUE(user_id, sanity_id)  
);
```

user_learning_paths

Stores generated personalized learning curricula per user.

```
CREATE TABLE public.user_learning_paths (  
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  user_id     UUID NOT NULL REFERENCES auth.users(id) ON DELETE CASCADE,  
  path_data   JSONB NOT NULL,    -- Full curriculum JSON (weeks, items, etc.)  
  created_at  TIMESTAMPTZ DEFAULT NOW() NOT NULL,  
  updated_at  TIMESTAMPTZ DEFAULT NOW() NOT NULL  
);
```

survey_aggregate

Stores aggregate survey results published as platform intelligence content.

```
CREATE TABLE public.survey_aggregate (  
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  edition     TEXT,  
  results     JSONB,  
  created_at  TIMESTAMPTZ DEFAULT NOW()  
);
```

ticker_cache

Caches news headlines for the ticker strip to avoid hammering external sources.

```
CREATE TABLE public.ticker_cache (  
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  headlines   JSONB NOT NULL,    -- Array of {text, url}  
  created_at  TIMESTAMPTZ DEFAULT NOW() NOT NULL  
);
```

5. Schema — Community & Social

referrals

Tracks referral links for growth tracking.

```
CREATE TABLE public.referrals (  
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  referrer_id UUID REFERENCES auth.users(id),  
  referee_id  UUID REFERENCES auth.users(id),  
  code       TEXT UNIQUE,  
  created_at  TIMESTAMPTZ DEFAULT NOW() NOT NULL  
);
```

role_change_log

Audit trail for all role grants and revocations.

```
CREATE TABLE public.role_change_log (  
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  user_id     UUID REFERENCES auth.users(id),  
  changed_by  UUID REFERENCES auth.users(id),  
  old_role    TEXT,  
  new_role    TEXT,  
  reason      TEXT,  
  created_at  TIMESTAMPTZ DEFAULT NOW() NOT NULL  
);
```

6. Row Level Security Policies

All user-data tables have RLS enabled. The service role key (used by the backend and Next.js API routes) bypasses RLS entirely.

Pattern 1 — Own data only

```
-- profiles: users see and update their own profile  
CREATE POLICY "Users can view own profile"  
ON profiles FOR SELECT  
USING (auth.uid() = id);  
  
CREATE POLICY "Users can update own profile"  
ON profiles FOR UPDATE  
USING (auth.uid() = id);
```

Pattern 2 — Admin override

```
-- user_roles: users see their own, admins see all  
CREATE POLICY "Users can view own roles"  
ON user_roles FOR SELECT  
USING (  
  auth.uid() = user_id  
  OR public.has_role(auth.uid(), 'admin')  
);
```

Pattern 3 — Service role for write operations

Role grants, subscription updates, and webhook processing use the service role key from Next.js API routes — these bypass RLS. Never expose the service role key to the browser.

Tables With RLS Enabled

Table	Read Policy	Write Policy
profiles	Own row	Own row
user_roles	Own rows	Service role only
subscriptions	Own row	Service role only
api_keys	Own rows	Own rows
bookmarks	Own rows	Own rows
user_learning_paths	Own rows	Own rows
conversations	Own rows	Own rows
conversation_messages	Via conversation	Via conversation

Tables Without RLS (Service Role Access Only)

- rag_documents — write via backend ingest process only
- rag_query_log — write via backend, admin read via dashboard
- state_performance_index — read public, write via admin only
- rht_state_profiles — read public, write via admin only
- stripe_events — write via webhook handler only
- webhook_inbox — write via webhook handler only
- role_change_log — service role only
- ingest_jobs — backend only

7. Database Functions & Triggers

handle_new_user() — Auto-provision on signup

```
CREATE OR REPLACE FUNCTION public.handle_new_user()
RETURNS TRIGGER LANGUAGE plpgsql SECURITY DEFINER AS $$
BEGIN
  -- Create profile
  INSERT INTO public.profiles (id, email, full_name, avatar_url)
  VALUES (NEW.id, NEW.email,
    COALESCE(NEW.raw_user_meta_data->>'full_name', ''),
    COALESCE(NEW.raw_user_meta_data->>'avatar_url', ''))
  ON CONFLICT (id) DO NOTHING;

  -- Assign default 'free' role
  INSERT INTO public.user_roles (user_id, role)
  VALUES (NEW.id, 'free')
  ON CONFLICT (user_id, role) DO NOTHING;

  -- Create subscription record
  INSERT INTO public.subscriptions (user_id, plan, status)
  VALUES (NEW.id, 'free', 'active')
  ON CONFLICT (user_id) DO NOTHING;

  RETURN NEW;
END;
$$;

CREATE TRIGGER on_auth_user_created
AFTER INSERT ON auth.users
FOR EACH ROW EXECUTE FUNCTION public.handle_new_user();
```


get_user_role(uid) — Highest role lookup

Returns the highest role held by a user.

```
CREATE OR REPLACE FUNCTION public.get_user_role(uid UUID)
RETURNS TEXT LANGUAGE plpgsql SECURITY DEFINER AS $$
DECLARE v_role TEXT;
BEGIN
  SELECT role::TEXT INTO v_role
  FROM public.user_roles
  WHERE user_id = uid
  ORDER BY
    CASE role::TEXT
      WHEN 'admin'          THEN 1
      WHEN 'advisory'       THEN 2
      WHEN 'professional'   THEN 3
      WHEN 'student'        THEN 4
      WHEN 'subscriber'     THEN 5
      ELSE 6
    END
  LIMIT 1;
  RETURN COALESCE(v_role, 'free');
END;
$;
```

has_role(uid, required_role) — Role check

Returns **TRUE** if the user holds the specified role. Admins always return **TRUE**.

```
CREATE OR REPLACE FUNCTION public.has_role(uid UUID, required_role TEXT)
RETURNS BOOLEAN LANGUAGE plpgsql SECURITY DEFINER AS $$
BEGIN
  IF EXISTS (
    SELECT 1 FROM public.user_roles WHERE user_id = uid AND role = 'admin'
  ) THEN RETURN TRUE; END IF;

  RETURN EXISTS (
    SELECT 1 FROM public.user_roles WHERE user_id = uid AND role::TEXT =
required_role
  );
END;
$;
```

set_updated_at() — Auto-update trigger

Applied to all tables with an **updated_at** column.

```
CREATE OR REPLACE FUNCTION public.set_updated_at()
RETURNS TRIGGER LANGUAGE plpgsql AS $$
BEGIN
  NEW.updated_at = NOW();
  RETURN NEW;
END;
$;
```

8. Hybrid Search Implementation

The AI Analyst uses a hybrid BM25 + vector search merged with Reciprocal Rank Fusion (RRF). This is implemented as a PostgreSQL stored function.

hybrid_search_rag RPC

```
CREATE OR REPLACE FUNCTION public.hybrid_search_rag(
  query_embedding VECTOR(768),
  query_text      TEXT,
  match_count     INT DEFAULT 20
)
RETURNS TABLE (
  id          UUID,
  chunk_text  TEXT,
  metadata    JSONB,
  rrf_score   FLOAT
)
LANGUAGE plpgsql AS $$
DECLARE
  dense_weight FLOAT := 0.6;
  sparse_weight FLOAT := 0.4;
BEGIN
  RETURN QUERY
  WITH dense AS (
    SELECT
      r.id,
      r.chunk_text,
      r.metadata,
      1 - (r.embedding <=> query_embedding) AS score,
      ROW_NUMBER() OVER (ORDER BY r.embedding <=> query_embedding) AS rank
    FROM public.rag_documents r
    WHERE r.embedding IS NOT NULL
    LIMIT match_count * 2
  ),
  sparse AS (
    SELECT
      r.id,
      r.chunk_text,
      r.metadata,
      ts_rank(to_tsvector('english', r.chunk_text),
        plainto_tsquery('english', query_text)) AS score,
      ROW_NUMBER() OVER (
        ORDER BY ts_rank(to_tsvector('english', r.chunk_text),
          plainto_tsquery('english', query_text)) DESC
      ) AS rank
    FROM public.rag_documents r
    WHERE to_tsvector('english', r.chunk_text)
      @@ plainto_tsquery('english', query_text)
    LIMIT match_count * 2
  ),
  rrf AS (
    SELECT
      COALESCE(d.id, s.id) AS id,
      COALESCE(d.chunk_text, s.chunk_text) AS chunk_text,
      COALESCE(d.metadata, s.metadata) AS metadata,
      (
        COALESCE(dense_weight / (60.0 + d.rank), 0) +
        COALESCE(sparse_weight / (60.0 + s.rank), 0)
      ) AS rrf_score
    FROM dense d
    FULL OUTER JOIN sparse s ON d.id = s.id
  )
  SELECT rrf.id, rrf.chunk_text, rrf.metadata, rrf.rrf_score
  FROM rrf
  ORDER BY rrf.rrf_score DESC
  LIMIT match_count;
```

```
END;  
$$;
```

How it works:

- 1. **Dense retrieval** — embeds the query using `text-embedding-3-small`, finds cosine-nearest chunks via the HNSW index
- 2. **Sparse retrieval** — tokenizes the query into a `tsquery`, ranks chunks by `ts_rank`
- 3. **RRF merge** — fuses both ranked lists using the formula `weight / (60 + rank)`, favoring chunks that appear high in both lists
- 4. **Weighting** — dense gets 0.6, sparse gets 0.4 (tunable in the function)

The Python `HybridRetriever` class in `backend/services/retrieval.py` calls this RPC, then passes the top-20 results to FlashRank for cross-encoder re-ranking before returning the top-5 to the LLM.

9. Migrations

Migrations live in `supabase/migrations/` and are numbered sequentially.

Migration	Contents
<code>001_profiles_and_roles.sql</code>	<code>profiles</code> , <code>user_roles</code> , <code>subscriptions</code> , <code>stripe_customers</code> , <code>stripe_events</code> , role enums, <code>handle_new_user</code> trigger
<code>002_content_data.sql</code>	<code>state_performance_index</code> , <code>rht_state_profiles</code> , <code>survey_aggregate</code>
<code>003_academy.sql</code>	Academy enrollment, progress, certificate tables
<code>004_advisory.sql</code>	Advisory engagement intake tables
<code>005_rag_vectors.sql</code>	<code>rag_documents</code> , HNSW index, <code>match_rag_documents</code> function
<code>006_rls_policies.sql</code>	All Row Level Security policies
<code>007_hybrid_search.sql</code>	<code>hybrid_search_rag</code> RPC function
<code>008_rls_audit.sql</code>	RLS audit logging
<code>009_referrals.sql</code>	<code>referrals</code> table
<code>010_community.sql</code>	Community and discussion tables
<code>011_api_keys.sql</code>	<code>api_keys</code> table
<code>012_survey.sql</code>	Survey response tables
<code>013_role_audit.sql</code>	<code>role_change_log</code> table
<code>014_webhook_inbox.sql</code>	<code>webhook_inbox</code> table
<code>015_rag_query_log.sql</code>	<code>rag_query_log</code> table
<code>016_bookmarks.sql</code>	<code>bookmarks</code> table
<code>017_ticker_cache.sql</code>	<code>ticker_cache</code> table
<code>018_role_change_log.sql</code>	Trigger for automatic role change logging
<code>019_user_learning_paths.sql</code>	<code>user_learning_paths</code> table

Running Migrations

Migrations run in Supabase's SQL editor or via the CLI:

```
# Using Supabase CLI  
supabase db push
```

```
# Or manually in the SQL editor – paste each file in order
```

Run migrations in numerical order. Each migration uses `IF NOT EXISTS` clauses and `DO $$ BEGIN ... EXCEPTION WHEN duplicate_object THEN NULL; END $$` blocks for idempotency.

10. Supabase Client Patterns

Server Components (SSR)

Use the SSR client from `@supabase/ssr` — reads cookies to authenticate the user.

```
// frontend/lib/supabase/server.ts
import { createServerClient } from "@supabase/ssr";
import { cookies } from "next/headers";

export function createClient() {
  const cookieStore = cookies();
  return createServerClient(
    process.env.NEXT_PUBLIC_SUPABASE_URL!,
    process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY!,
    {
      cookies: {
        getAll: () => cookieStore.getAll(),
        setAll: (cookiesToSet) => {
          cookiesToSet.forEach(({ name, value, options }) =>
            cookieStore.set(name, value, options)
          );
        },
      },
    },
  );
}
```

API Routes (Service Role)

Use the service role key for writes and admin operations — bypasses RLS.

```
// frontend/lib/supabase/service.ts
import { createClient as createSupabaseClient } from "@supabase/supabase-js";

export function createServiceClient() {
  return createSupabaseClient(
    process.env.NEXT_PUBLIC_SUPABASE_URL!,
    process.env.SUPABASE_SERVICE_ROLE_KEY!
  );
}
```

Backend (Python)

```
# backend/services/db.py
from supabase import create_client, Client
from config import SUPABASE_URL, SUPABASE_SERVICE_ROLE_KEY

_supabase: Client | None = None

def get_supabase() -> Client:
    global _supabase
```

```
if _supabase is None:
    _supabase = create_client(SUPABASE_URL, SUPABASE_SERVICE_ROLE_KEY)
return _supabase
```

Common Query Patterns

```
// Fetch user's role
const { data: roles } = await supabase
  .from("user_roles")
  .select("role")
  .eq("user_id", userId);

// Upsert a bookmark
await supabase.from("bookmarks").upsert({
  user_id: userId,
  sanity_id: sanityId,
  slug,
  title,
  pillar,
}, { onConflict: "user_id,sanity_id" });

// Delete a row (own data)
await supabase.from("bookmarks")
  .delete()
  .eq("user_id", userId)
  .eq("sanity_id", sanityId);

// Paginated query
const { data, count } = await supabase
  .from("rag_query_log")
  .select("*, { count: "exact" })
  .order("created_at", { ascending: false })
  .range(0, 19); // first 20 rows

// RPC call
const { data } = await supabase.rpc("hybrid_search_rag", {
  query_embedding: embedding,
  query_text: queryText,
  match_count: 20,
});
```

11. Performance & Indexing

Key Indexes

Table	Column(s)	Type	Purpose
rag_documents	embedding	HNSW (cosine)	ANN vector search
rag_documents	chunk_text (tsvector)	GIN	Full-text BM25 search
rag_documents	source_type, source_id	B-tree	Incremental re-indexing
rag_documents	pillar	B-tree	Pillar-filtered retrieval
user_roles	user_id	B-tree (via UNIQUE)	Role lookups in middleware
bookmarks	user_id, sanity_id	B-tree (via UNIQUE)	Bookmark dedup
subscriptions	user_id	B-tree (via UNIQUE)	Subscription lookup

Connection Pooling

Supabase uses PgBouncer in transaction mode. The backend Python service uses the direct database URL (`SUPABASE_DB_URL`) for pgvector writes during ingestion, which bypasses PgBouncer for better compatibility with pgvector operations. All other queries go through the Supabase REST API.

HNSW Index Tuning

The current HNSW index uses:

- `m = 16` — maximum connections per node (higher = more accurate, slower build)
- `ef_construction = 64` — candidate list size at build time (higher = more accurate)

For larger document volumes (>100k chunks), increase `ef_construction` to 128 and rebuild the index:

```
DROP INDEX idx_rag_embedding;
CREATE INDEX idx_rag_embedding
  ON public.rag_documents
  USING hsw (embedding vector_cosine_ops)
  WITH (m = 16, ef_construction = 128);
```

12. Backup & Recovery

Automatic Backups

Supabase Pro and above plans include daily automated backups with point-in-time recovery (PITR). Retention is 7 days on Pro, 30 days on Enterprise.

Manual Export

```
# Export schema + data
pg_dump "postgresql://postgres:[password]@db.
[project].supabase.co:5432/postgres" \
  --no-owner --no-acl -f backup.sql

# Export data only (skip migrations)
pg_dump "postgresql://..." --data-only -f data-only.sql
```

Critical Tables to Backup Separately

- `rag_documents` — Large; re-ingestion from Sanity is possible but takes ~10 minutes
- `state_performance_index` — Seeded from static data; re-seeding from `supabase/seed/` is possible
- `subscriptions` + `stripe_events` — Financial data; back up before any schema changes

Recovery Priority Order

1. `auth.users` — Core user accounts (managed by Supabase, backed up automatically)
2. `profiles`, `user_roles`, `subscriptions` — User state
3. `state_performance_index`, `rht_state_profiles` — Platform data
4. `rag_documents` — Recoverable via re-ingest from Sanity